

AI Transit Pipeline

Installation Guide

Version 2.0 · 2026-08-24

Applicable to: `fetch_repo.sh` · `scan_pipeline.sh` · `ai_transit.sh` · `generate_excel_report.py` · `selfcheck.py`

Table of Contents

- Overview
- Prerequisites
- Core System Packages
- Python Environment
- Tool-by-Tool Installation
 - 5.1 betterleaks — secret detection (L1)
 - 5.2 detect-secrets — entropy scanning (L1)
 - 5.3 ClamAV — antivirus (L1)
 - 5.4 YARA — custom IOC rules (L1)
 - 5.5 Semgrep — OWASP / CWE / CERT (L2)
 - 5.6 trivy — SCA / CVE (L3)
 - 5.7 pip-audit — Python CVE (L3)
 - 5.8 safety — Python advisories (L3)
 - 5.9 npm + npm audit — Node.js CVE (L3)
 - 5.10 Bandit — Python SAST (L5)
 - 5.11 ShellCheck — shell SAST (L5)
 - 5.12 cppcheck — C/C++ SAST (L5)
 - 5.13 hadolint — Dockerfile linter (L5)
 - 5.14 checkov — Terraform / IaC (L5)
 - 5.15 ScanCode Toolkit — licence & copyright (L6)
- Pipeline Scripts Installation
- Directory Structure Setup
- Environment Variables
- Full Installation Verification
- Offline Operation
- Sample Scans — Testing the Pipeline
- Self-Scan: Verifying the Installation is Safe
- Security Hardening Recommendations
- Troubleshooting

1. Overview

The AI Transit Pipeline is a 6-layer security gateway that scans AI-generated code before enterprise import.

Layer	Tools	Blocks on
L1 — Secrets & AV	betterleaks, detect-secrets, ClamAV, YARA	Secret/credential/malware detected
L2 — OWASP/CWE	Semgrep	ERROR/WARNING severity finding
L3 — SCA/CVE	trivy, pip-audit, safety, npm audit	HIGH/CRITICAL CVE in dependency
L4 — Patterns	grep (built-in)	CWE-798/22/918/327/338 match
L5 — Per-type SAST	Bandit, ShellCheck, cppcheck, hadolint, checkov	Tool-specific finding
L6 — Licence	ScanCode Toolkit	CRITICAL/HIGH CVE in detected package

Verdict: any FAIL in any layer → package quarantined. WARNs are logged but do not block.

2. Prerequisites

Supported platforms: Ubuntu 22.04 LTS / Debian 12 (production), Ubuntu 24.04 LTS (recommended).

Windows users: install via **WSL2** (see Section 4 of base guide).

Minimum hardware: 4-core CPU, 8 GB RAM, 20 GB free disk.

3. Core System Packages

Install the baseline packages first. These are required for the pipeline to run at all.

```
sudo apt-get update && sudo apt-get upgrade -y

sudo apt-get install -y \
bash git curl ca-certificates wget gpg \
jq zip unzip file coreutils \
python3 python3-pip python3-venv \
build-essential golang-go \
nodejs npm
```

Verify:

```
bash --version | head -1 # expect: GNU bash, version 5.x
git --version # expect: git version 2.x
python3 --version # expect: Python 3.10+
go version # expect: go1.21+
node --version # expect: v18+ or v20+
jq --version # expect: jq-1.6+
```

4. Python Virtual Environment

Using a dedicated venv avoids conflicts with system packages and makes the installation self-contained.

```
python3 -m venv /opt/ai-transit/venv
echo 'source /opt/ai-transit/venv/bin/activate' >> ~/.bashrc
source /opt/ai-transit/venv/bin/activate
pip install --upgrade pip
```

All pip install commands in this guide assume the venv is active.

Verify:

```
which python3 # must show /opt/ai-transit/venv/bin/python3
python3 --version # Python 3.10+
```

5. Tool-by-Tool Installation

Each section covers: finding the latest version, installing, verifying, offline setup, and a functional test.

5.1 betterleaks — Secret Detection (Layer 1) {#betterleaks}

What it does: scans all file types for secrets, API keys, credentials and tokens. Successor to gitleaks with a more expressive allowlist system.

Find the latest version

```
# Query GitHub API for latest release tag
curl -s https://api.github.com/repos/betterleaks/betterleaks/releases/latest \
| jq -r '.tag_name'
```

Install (Go — recommended, produces a static binary)

```
go install github.com/betterleaks/betterleaks@latest
sudo cp ~/go/bin/betterleaks /usr/local/bin/betterleaks
```

Install (Homebrew — macOS / Linux with brew)

```
brew install betterleaks
```

Install (Fedora / RHEL)

```
sudo dnf install betterleaks
```

Verify installation

```
betterleaks --version
# Expected output: betterleaks version x.y.z
```

Offline operation

betterleaks has **no external database** — all detection rules are embedded in the binary. Once installed, it works fully offline.

Functional test

```
# Create a test file with a fake secret
mkdir /tmp/bl-test
echo 'AWS_SECRET_ACCESS_KEY="wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"' \
> /tmp/bl-test/config.py

betterleaks dir /tmp/bl-test -v
# Expected: non-zero exit code, finding reported for config.py

# Clean file — should PASS
echo 'import os; key = os.environ["AWS_SECRET_ACCESS_KEY"]' \
> /tmp/bl-test/config_clean.py
betterleaks dir /tmp/bl-test/config_clean.py -v
# Expected: exit code 0

rm -rf /tmp/bl-test
```

5.2 detect-secrets — Entropy Scanning (Layer 1) {#detect-secrets}

What it does: complements betterleaks with Shannon entropy analysis and regex-based patterns to find high-entropy strings that look like secrets.

Find the latest version

```
pip index versions detect-secrets 2>/dev/null | head -1
# or:
curl -s https://pypi.org/pypi/detect-secrets/json | jq -r '.info.version'
```

Install

```
pip install detect-secrets
```

Verify installation

```
detect-secrets --version
# Expected: x.y.z
```

Offline operation

No external database. Works fully offline once installed.

Functional test

```
mkdir /tmp/ds-test
echo 'password = "Sup3rS3cr3tP@ssw0rd!"' > /tmp/ds-test/app.py

detect-secrets scan /tmp/ds-test/app.py
# Expected: JSON with non-empty "results" containing app.py

echo 'password = os.getenv("PASSWORD")' > /tmp/ds-test/app_clean.py
detect-secrets scan /tmp/ds-test/app_clean.py
# Expected: JSON with empty "results" {}

rm -rf /tmp/ds-test
```

5.3 ClamAV — Antivirus (Layer 1) {#clamav}

What it does: scans files against a database of known malware signatures. Requires regular DB updates to stay effective.

Find the latest version

```
apt-cache policy clamav | grep Candidate
# or for the latest upstream:
curl -s https://www.clamav.net/downloads | grep -oP 'clamav-\K[0-9.]+(?:=\.tar)' | head -1
```

Install

```
sudo apt-get install -y clamav clamav-daemon clamav-freshclam
```

Update virus database (required before first use)

```
sudo systemctl stop clamav-freshclam
```

```
sudo freshclam
sudo systemctl start clamav-freshclam
sudo systemctl enable clamav-freshclam
```

Verify installation

```
clamscan --version
# Expected: ClamAV x.y.z/NNNNN/...
freshclam --version
# Expected: ClamAV x.y.z
```

Offline operation

ClamAV works offline once the virus DB is downloaded. To prepare for offline use:

```
# On a machine with internet access, download the DB files
sudo freshclam

# The DB files are stored at (copy these to the offline machine):
ls /var/lib/clamav/
# main.cvd (or main.cld), daily.cvd (or daily.cld), bytecode.cvd

# On the offline machine, copy the DB files to /var/lib/clamav/
# then reload:
sudo systemctl restart clamav-daemon
```

Functional test

```
# ClamAV includes the EICAR test signature – a safe test virus string
echo 'X50!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*' \
> /tmp/eicar.txt

clamscan /tmp/eicar.txt
# Expected: /tmp/eicar.txt: Eicar-Signature FOUND
# Expected exit code: 1

clamscan /bin/ls
# Expected: /bin/ls: OK
# Expected exit code: 0

rm /tmp/eicar.txt
```

5.4 YARA — Custom IOC Rules (Layer 1) {#yara}

What it does: loads organisation-specific YARA rules from yara-rules/ and matches them against every file. Fully customisable pattern engine.

Find the latest version

```
apt-cache policy yara | grep Candidate
# or from GitHub:
curl -s https://api.github.com/repos/VirusTotal/yara/releases/latest \
| jq -r '.tag_name'
```

Install

```
sudo apt-get install -y yara
```

Verify installation

```
yara --version  
# Expected: x.y.z
```

Offline operation

YARA has no external database — all rules are local .yar files. Works fully offline.

Create a minimal test rule

```
mkdir -p /opt/ai-transit/yara-rules  
  
cat > /opt/ai-transit/yara-rules/test.yar << 'EOF'  
rule Suspicious_Base64_Exec {  
  meta:  
    description = "Detects base64-encoded exec calls"  
    strings:  
      $b64_exec = /eval\(base64_decode\(/ nocase  
    condition:  
      $b64_exec  
    }  
  }  
EOF
```

Functional test

```
echo '<?php eval(base64_decode("dw5saw5r...")); ?>' > /tmp/test.php  
  
yara /opt/ai-transit/yara-rules/test.yar /tmp/test.php  
# Expected: Suspicious_Base64_Exec /tmp/test.php  
  
echo '<?php echo "Hello World"; ?>' > /tmp/clean.php  
yara /opt/ai-transit/yara-rules/test.yar /tmp/clean.php  
# Expected: (no output – no match)  
  
rm /tmp/test.php /tmp/clean.php
```

5.5 Semgrep — OWASP / CWE / CERT Static Analysis (Layer 2) {#semgrep}

What it does: runs four security rulesets (OWASP Top 10, CWE Top 25, security-audit, secrets) across all supported languages using pattern matching on the AST.

Find the latest version

```
curl -s https://pypi.org/pypi/semgrep/json | jq -r '.info.version'
```

Install

```
pip install semgrep
```

Verify installation

```
semgrep --version  
# Expected: 1.x.x
```

Offline operation

Semgrep needs internet access **only on the first run** to download rules. Prepare for offline:

```
# On a machine with internet, pre-download rules to a local directory
mkdir -p /opt/ai-transit/semgrep-rules
semgrep --config p/owasp-top-ten --dump-config > /opt/ai-transit/semgrep-rules/owasp.yaml
semgrep --config p/cwe-top-25 --dump-config > /opt/ai-transit/semgrep-rules/cwe.yaml
semgrep --config p/security-audit --dump-config > /opt/ai-transit/semgrep-rules/audit.yaml
semgrep --config p/secrets --dump-config > /opt/ai-transit/semgrep-rules/secrets.yaml

# On the offline machine, run against local rules:
semgrep --config /opt/ai-transit/semgrep-rules/ <repo_dir>
```

Functional test

```
cat > /tmp/test_sqli.py << 'EOF'
import sqlite3
def get_user(username):
    conn = sqlite3.connect("db.sqlite3")
    query = "SELECT * FROM users WHERE name = '" + username + "'"
    return conn.execute(query).fetchall()
EOF

semgrep --config p/owasp-top-ten --json /tmp/test_sqli.py \
| jq '.results | length'
# Expected: 1 or more (SQL injection detected)

cat > /tmp/test_clean.py << 'EOF'
import sqlite3
def get_user(username):
    conn = sqlite3.connect("db.sqlite3")
    return conn.execute("SELECT * FROM users WHERE name = ?", (username,)).fetchall()
EOF

semgrep --config p/owasp-top-ten --json /tmp/test_clean.py \
| jq '.results | length'
# Expected: 0

rm /tmp/test_sqli.py /tmp/test_clean.py
```

5.6 trivy — SCA / CVE (Layer 3) {#trivy}

What it does: scans dependency manifests (requirements.txt, package-lock.json, go.sum, pom.xml, Cargo.lock ...) against NVD, OSV and GitHub Advisory databases.

Find the latest version

```
curl -s https://api.github.com/repos/aquasecurity/trivy/releases/latest \
| jq -r '.tag_name'
```

Install (official script — recommended)

```
curl -sL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh \
```

```
| sudo sh -s -- -b /usr/local/bin
```

Install (apt repository)

```
sudo apt-get install -y wget gnupg
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key \
| sudo gpg --dearmor -o /usr/share/keyrings/trivy.gpg
echo "deb [signed-by=/usr/share/keyrings/trivy.gpg] \
https://aquasecurity.github.io/trivy-repo/deb generic main" \
| sudo tee /etc/apt/sources.list.d/trivy.list
sudo apt-get update && sudo apt-get install -y trivy
```

Verify installation

```
trivy --version
# Expected: Version: x.y.z
```

Download vulnerability database (required before first offline use)

```
# Pre-download the CVE database (requires internet)
trivy image --download-db-only
trivy image --download-java-db-only

# DB is cached at:
ls ~/.cache/trivy/db/
```

Offline operation

```
# After downloading the DB, run offline:
trivy fs <repo_dir> \
--scanners vuln \
--severity HIGH,CRITICAL \
--skip-db-update \
--offline-scan
```

Functional test

```
mkdir /tmp/trivy-test

# Create a requirements.txt with a known vulnerable package
cat > /tmp/trivy-test/requirements.txt << 'EOF'
Pillow==9.0.0
requests==2.18.0
EOF

trivy fs /tmp/trivy-test \
--scanners vuln \
--severity HIGH,CRITICAL \
--format table
# Expected: findings for known CVEs in Pillow 9.0.0 and requests 2.18.0

# Clean requirements
cat > /tmp/trivy-test/requirements_clean.txt << 'EOF'
Pillow==10.3.0
requests==2.32.3
```

```
EOF
```

```
trivy fs /tmp/trivy-test/requirements_clean.txt \  
--scanners vuln \  
--severity HIGH,CRITICAL \  
--format table  
# Expected: No vulnerabilities found  
  
rm -rf /tmp/trivy-test
```

5.7 pip-audit — Python CVE (Layer 3) {#pip-audit}

What it does: audits Python requirements files against the Python Packaging Advisory Database (PyPA) and OSV.

Find the latest version

```
curl -s https://pypi.org/pypi/pip-audit/json | jq -r '.info.version'
```

Install

```
pip install pip-audit
```

Verify installation

```
pip-audit --version  
# Expected: pip-audit x.y.z
```

Offline operation

pip-audit requires internet to query the PyPA/OSV database. For offline use, use trivy with `--offline-scan` instead (covers the same packages).

Functional test

```
cat > /tmp/req_vuln.txt << 'EOF'  
Pillow==9.0.0  
requests==2.18.0  
EOF  
  
pip-audit -r /tmp/req_vuln.txt  
# Expected: one or more vulnerabilities listed, exit code 1  
  
cat > /tmp/req_clean.txt << 'EOF'  
Pillow==10.3.0  
requests==2.32.3  
EOF  
  
pip-audit -r /tmp/req_clean.txt  
# Expected: No known vulnerabilities found, exit code 0  
  
rm /tmp/req_vuln.txt /tmp/req_clean.txt
```

5.8 safety — Python Advisories (Layer 3) {#safety}

What it does: checks Python dependencies against the Safety DB (PyUp.io), a curated advisory database with additional entries not always in OSV.

Find the latest version

```
curl -s https://pypi.org/pypi/safety/json | jq -r '.info.version'
```

Install

```
pip install safety
```

Verify installation

```
safety --version  
# Expected: safety, version x.y.z
```

Offline operation

safety requires internet for DB queries. For offline environments, use trivy as the primary SCA scanner.

Functional test

```
cat > /tmp/req_vuln.txt << 'EOF'  
Pillow==9.0.0  
EOF  
  
safety check -r /tmp/req_vuln.txt  
# Expected: vulnerabilities found, exit code 64  
  
cat > /tmp/req_clean.txt << 'EOF'  
Pillow==10.3.0  
EOF  
  
safety check -r /tmp/req_clean.txt  
# Expected: No known security vulnerabilities found, exit code 0  
  
rm /tmp/req_vuln.txt /tmp/req_clean.txt
```

5.9 npm + npm audit — Node.js CVE (Layer 3) {#npm-audit}

What it does: built into npm — audits package-lock.json against the npm security advisory registry for known CVEs in Node.js dependencies.

Find the latest version

```
npm --version  
node --version  
# npm is bundled with Node.js; update both together
```

Install (via NodeSource — LTS)

```
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -  
sudo apt-get install -y nodejs
```

Verify installation

```
node --version # Expected: v20.x.x or v22.x.x
```

```
npm --version # Expected: 10.x.x or later
npm audit --version 2>/dev/null || echo "npm audit is built-in"
```

Offline operation

npm audit requires internet to query the npm advisory registry. No offline mode is available.

Functional test

```
mkdir /tmp/npm-test && cd /tmp/npm-test

# Create a package.json with a known vulnerable package
cat > package.json << 'EOF'
{
  "name": "test",
  "version": "1.0.0",
  "dependencies": {
    "lodash": "4.17.4"
  }
}
EOF

npm install --package-lock-only 2>/dev/null
npm audit --json | jq '.metadata.vulnerabilities'
# Expected: object with non-zero high/critical counts

cd / && rm -rf /tmp/npm-test
```

5.10 Bandit — Python SAST (Layer 5) {#bandit}

What it does: Python-specific static analyser with 100+ plugins detecting SQL injection, shell injection, hardcoded passwords, weak crypto, insecure deserialization, and more.

Find the latest version

```
curl -s https://pypi.org/pypi/bandit/json | jq -r '.info.version'
```

Install

```
pip install bandit
```

Verify installation

```
bandit --version
# Expected: bandit x.y.z (Python x.y.z)
```

Offline operation

Bandit has no external database. Works fully offline.

Functional test

```
cat > /tmp/test_bandit.py << 'EOF'
import subprocess
import hashlib

# B602 - subprocess with shell=True
```

```
def run(cmd):
    subprocess.call(cmd, shell=True)

# B324 - use of weak hash
def hash_password(pwd):
    return hashlib.md5(pwd.encode()).hexdigest()
EOF

bandit -ll /tmp/test_bandit.py
# Expected: Issue: [B602] ... [B324] ... exit code 1

cat > /tmp/test_bandit_clean.py << 'EOF'
import subprocess
import hashlib

def run(cmd_list):
    subprocess.call(cmd_list)

def hash_password(pwd):
    return hashlib.sha256(pwd.encode()).hexdigest()
EOF

bandit -ll /tmp/test_bandit_clean.py
# Expected: No issues identified, exit code 0

rm /tmp/test_bandit.py /tmp/test_bandit_clean.py
```

5.11 ShellCheck — Shell Script SAST (Layer 5) {#shellcheck}

What it does: static analyser for bash/sh/dash/ksh. Detects quoting errors, command injection risks, undefined variables, deprecated syntax, and unsafe patterns.

Find the latest version

```
apt-cache policy shellcheck | grep Candidate
# or latest binary from GitHub:
curl -s https://api.github.com/repos/koalaman/shellcheck/releases/latest \
| jq -r '.tag_name'
```

Install (apt)

```
sudo apt-get install -y shellcheck
```

Install (latest binary from GitHub)

```
VERSION=$(curl -s https://api.github.com/repos/koalaman/shellcheck/releases/latest \
| jq -r '.tag_name')
curl -sSL "https://github.com/koalaman/shellcheck/releases/download/${VERSION}/shellcheck-${VERSION}.linux.x86_64.tar.xz" \
| tar -xJ --strip-components=1 -C /tmp
sudo mv /tmp/shellcheck /usr/local/bin/shellcheck
```

Verify installation

```
shellcheck --version
```

```
# Expected: version: x.y.z
```

Offline operation

ShellCheck has no external database. Works fully offline.

Functional test

```
cat > /tmp/test.sh << 'EOF'
#!/bin/bash
filename=$1
if [ $filename == "admin" ]; then
echo "Hello $filename"
fi
rm -rf /$filename
EOF

shellcheck --severity=warning /tmp/test.sh
# Expected: SC2086 (unquoted variable), SC2115 (unsafe rm), exit code 1

cat > /tmp/test_clean.sh << 'EOF'
#!/bin/bash
filename="$1"
if [[ "$filename" == "admin" ]]; then
echo "Hello $filename"
fi
EOF

shellcheck --severity=warning /tmp/test_clean.sh
# Expected: exit code 0

rm /tmp/test.sh /tmp/test_clean.sh
```

5.12 cppcheck — C/C++ SAST (Layer 5) {#cppcheck}

What it does: static analyser for C and C++ detecting buffer overflows, memory leaks, null-pointer dereferences, use-after-free, and undefined behaviour without compiling.

Find the latest version

```
apt-cache policy cppcheck | grep Candidate
# or from GitHub:
curl -s https://api.github.com/repos/danmar/cppcheck/releases/latest \
| jq -r '.tag_name'
```

Install (apt)

```
sudo apt-get install -y cppcheck
```

Verify installation

```
cppcheck --version
# Expected: Cppcheck x.y
```

Offline operation

cppcheck has no external database. Works fully offline.

Functional test

```
cat > /tmp/test.cpp << 'EOF'
#include <string.h>
void test() {
char buf[10];
strcpy(buf, "Hello, this string is too long and will overflow the buffer!");
int* p = new int(42);
delete p;
*p = 100; // use after free
}
EOF

cppcheck --enable=warning,security --error-exitcode=1 /tmp/test.cpp
# Expected: errors for buffer overflow and use-after-free, exit code 1

cat > /tmp/test_clean.cpp << 'EOF'
#include <cstring>
void test() {
char buf[100];
strncpy(buf, "Hello", sizeof(buf) - 1);
buf[sizeof(buf) - 1] = '\0';
}
EOF

cppcheck --enable=warning,security --error-exitcode=1 /tmp/test_clean.cpp
# Expected: no errors, exit code 0

rm /tmp/test.cpp /tmp/test_clean.cpp
```

5.13 hadolint — Dockerfile Linter (Layer 5) {#hadolint}

What it does: enforces Dockerfile best practices and detects security misconfigurations: running as root, :latest tags, baked-in secrets, ADD vs COPY, shell injection.

Find the latest version

```
curl -s https://api.github.com/repos/hadolint/hadolint/releases/latest \
| jq -r '.tag_name'
```

Install (binary)

```
VERSION=$(curl -s https://api.github.com/repos/hadolint/hadolint/releases/latest \
| jq -r '.tag_name')
curl -sSL "https://github.com/hadolint/hadolint/releases/download/${VERSION}/hadolint-Linux-x86_64" \
-o /usr/local/bin/hadolint
sudo chmod +x /usr/local/bin/hadolint
```

Verify installation

```
hadolint --version
# Expected: Haskell Dockerfile Linter x.y.z
```

Offline operation

hadolint has no external database. Works fully offline.

Functional test

```
cat > /tmp/Dockerfile_bad << 'EOF'
FROM ubuntu:latest
RUN apt-get update && apt-get install -y curl
ADD https://example.com/script.sh /tmp/
ENV PASSWORD=mysecretpassword
RUN /tmp/script.sh
EOF

hadolint /tmp/Dockerfile_bad
# Expected: DL3007 (latest tag), DL3009 (delete apt cache), SC2046, etc.

cat > /tmp/Dockerfile_good << 'EOF'
FROM ubuntu:22.04
RUN apt-get update \
&& apt-get install -y --no-install-recommends curl \
&& rm -rf /var/lib/apt/lists/*
COPY script.sh /tmp/script.sh
USER nobody
EOF

hadolint /tmp/Dockerfile_good
# Expected: no output or only style suggestions, exit code 0

rm /tmp/Dockerfile_bad /tmp/Dockerfile_good
```

5.14 checkov — Terraform / IaC SAST (Layer 5) {#checkov}

What it does: static analysis for Terraform, CloudFormation, Kubernetes YAML, Ansible and ARM templates. Maps findings to CIS Benchmarks, NIST, SOC2, and OWASP. 1000+ built-in checks.

Find the latest version

```
curl -s https://pypi.org/pypi/checkov/json | jq -r '.info.version'
```

Install

```
pip install checkov
```

Verify installation

```
checkov --version
# Expected: x.y.z
```

Offline operation

checkov uses local checks only — works fully offline. No external DB required.

Functional test

```
mkdir /tmp/tf-test

cat > /tmp/tf-test/main.tf << 'EOF'
resource "aws_s3_bucket" "public_bucket" {
```

```

bucket = "my-public-bucket"
acl = "public-read"
}

resource "aws_security_group" "open" {
  ingress {
    from_port = 22
    to_port = 22
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
EOF

checkov -d /tmp/tf-test --framework terraform --compact
# Expected: FAILED checks (public S3 bucket, open SSH to world)

cat > /tmp/tf-test/main_secure.tf << 'EOF'
resource "aws_s3_bucket" "private_bucket" {
  bucket = "my-private-bucket"
}
resource "aws_s3_bucket_acl" "private" {
  bucket = aws_s3_bucket.private_bucket.id
  acl = "private"
}
EOF

checkov -f /tmp/tf-test/main_secure.tf --framework terraform --compact
# Expected: all checks passed or significantly fewer failures

rm -rf /tmp/tf-test

```

5.15 ScanCode Toolkit — Licence & Copyright (Layer 6) {#scancode}

What it does: full-text licence detection using 30 000+ licence texts (SPDX), copyright notice extraction, package manifest detection, and CVE lookup in detected packages. Produces JSON/SPDX/CycloneDX reports.

Find the latest version

```
curl -s https://pypi.org/pypi/scancode-toolkit/json | jq -r '.info.version'
```

Install

```
pip install scancode-toolkit
```

ScanCode is a large package (~500 MB installed). Installation may take 5–10 minutes.

Verify installation

```
scancode --version
# Expected: ScanCode version x.y.z
```

Offline operation

ScanCode works **fully offline** — all licence texts and detection logic are bundled in the package. No external DB or internet required at scan time.

Functional test

```
mkdir /tmp/sc-test

# File with a GPL licence header
cat > /tmp/sc-test/app.py << 'EOF'
# This program is free software: you can redistribute it and/or modify
# it under the terms of the GNU General Public License as published by
# the Free Software Foundation, either version 3 of the License, or
# (at your option) any later version.
# Copyright (C) 2024 Example Corp.
import os

def main():
    print("hello")
EOF

# File with a permissive MIT licence
cat > /tmp/sc-test/lib.py << 'EOF'
# MIT License
# Copyright (c) 2024 Example Corp
# Permission is hereby granted, free of charge, to any person obtaining a copy...
def helper():
    pass
EOF

scancode --license --copyright \
--json-pp /tmp/sc-report.json \
--quiet \
/tmp/sc-test

# Check detections
jq '.files[] | {path: .path, licenses: [.license_detections[].matches[].spdx_license_expression]}' \
/tmp/sc-report.json

# Expected: GPL-3.0-or-later detected in app.py, MIT in lib.py

rm -rf /tmp/sc-test /tmp/sc-report.json
```

6. Pipeline Scripts Installation

6.1 Clone from GitHub

```
git clone https://github.com/gaillotte/claude.git
cd claude
git checkout claude/vigilant-carson-f8twy0
```

6.2 Install Python dependencies for report generation

```
pip install openpyxl reportlab
```

6.3 Make scripts executable

```
chmod +x ai_transit.sh fetch_repo.sh scan_pipeline.sh
```

6.4 Verify scripts

```
bash -n ai_transit.sh && echo "ai_transit.sh: syntax OK"
bash -n fetch_repo.sh && echo "fetch_repo.sh: syntax OK"
bash -n scan_pipeline.sh && echo "scan_pipeline.sh: syntax OK"
python3 -c "import ast; ast.parse(open('generate_excel_report.py').read())" \
&& echo "generate_excel_report.py: syntax OK"
python3 -c "import ast; ast.parse(open('selfcheck.py').read())" \
&& echo "selfcheck.py: syntax OK"
```

7. Directory Structure Setup

```
sudo mkdir -p /opt/ai-transit/{fetch,quarantine,approved,reports,logs,yara-rules}
sudo chmod 700 /opt/ai-transit/quarantine
sudo chown -R $USER:$USER /opt/ai-transit
mkdir -p Good
```

Copy your YARA rules to /opt/ai-transit/yara-rules/ (see §5.4 for a minimal test rule).

8. Environment Variables

Variable	Default	Description
WORK_DIR	/opt/ai-transit	Root working directory
OUTPUT_DIR	<script_dir>/Good	Output directory for approved ZIPs

Add to ~/.bashrc:

```
export WORK_DIR="/opt/ai-transit"
export OUTPUT_DIR="/opt/ai-transit/approved"
```

Or pass inline:

```
WORK_DIR=/data/ai-transit ./ai_transit.sh https://github.com/org/repo
```

9. Full Installation Verification

Save this as `verify_install.sh` and run it after completing all sections above:

```
#!/usr/bin/env bash
# verify_install.sh – checks all required and optional tools

PASS=0; WARN=0; FAIL=0

check() {
  local name="$1" cmd="$2"
  if command -v "$cmd" &>/dev/null; then
    echo -e "\033[32m[OK]\033[0m $name"
    (( PASS++ ))
  else
    echo -e "\033[33m[WARN]\033[0m $name – optional, not installed"
    (( WARN++ ))
  fi
}

require() {
  local name="$1" cmd="$2"
  if command -v "$cmd" &>/dev/null; then
    echo -e "\033[32m[OK]\033[0m $name"
    (( PASS++ ))
  else
    echo -e "\033[31m[FAIL]\033[0m $name – REQUIRED, not installed"
    (( FAIL++ ))
  fi
}

echo "=== Core system ==="
require "bash" bash
require "git" git
require "python3" python3
require "pip" pip
require "curl" curl
require "jq" jq
require "zip" zip
require "sha256sum" sha256sum
require "file" file

echo ""
echo "=== Python report packages ==="
python3 -c "import openpyxl" 2>/dev/null \
&& { echo -e "\033[32m[OK]\033[0m openpyxl"; (( PASS++ )); } \
|| { echo -e "\033[31m[FAIL]\033[0m openpyxl – pip install openpyxl"; (( FAIL++ )); }
python3 -c "import reportlab" 2>/dev/null \
&& { echo -e "\033[32m[OK]\033[0m reportlab"; (( PASS++ )); } \
|| { echo -e "\033[31m[FAIL]\033[0m reportlab – pip install reportlab"; (( FAIL++ )); }

echo ""
echo "=== Layer 1 – Secrets & AV ==="
check "betterleaks" betterleaks
check "detect-secrets" detect-secrets
check "ClamAV" clamscan
```

```

check "YARA" yara

echo ""
echo "=== Layer 2 – OWASP/CWE ==="
check "Semgrep" semgrep

echo ""
echo "=== Layer 3 – SCA/CVE ==="
check "trivy" trivy
check "pip-audit" pip-audit
check "safety" safety
check "npm" npm

echo ""
echo "=== Layer 5 – SAST ==="
check "Bandit" bandit
check "ShellCheck" shellcheck
check "cppcheck" cppcheck
check "hadolint" hadolint
check "checkov" checkov

echo ""
echo "=== Layer 6 – Licence ==="
check "scancode" scancode

echo ""
echo "=== Summary ==="
echo -e "\033[32mOK: $PASS\033[0m | \033[33mOptional missing: $WARN\033[0m | \033[31mRequired missing: $FAIL\033[0m"
[[ $FAIL -eq 0 ]] \
&& echo -e "\033[32mInstallation COMPLETE – pipeline ready\033[0m" \
|| echo -e "\033[31mInstallation INCOMPLETE – fix FAIL items above\033[0m"

```

```

chmod +x verify_install.sh && ./verify_install.sh

```

10. Offline Operation

The table below summarises offline readiness for each tool:

Tool	Offline?	Preparation needed
betterleaks	✓ Yes	None — rules embedded in binary
detect-secrets	✓ Yes	None — rules embedded
ClamAV	✓ Yes	Run <code>freshclam</code> on a connected machine, then copy <code>/var/lib/clamav/</code> DB files
YARA	✓ Yes	None — local <code>.yar</code> rule files only
Semgrep	✓ Yes*	Pre-download rules: <code>semgrep --config p/owasp-top-ten --dump-config > rules.yaml</code>

Tool	Offline?	Preparation needed
trivy	✓ Yes*	Run trivy image --download-db-only first; use --skip-db-update --offline-scan
pip-audit	✗ No	Use trivy offline as fallback
safety	✗ No	Use trivy offline as fallback
npm audit	✗ No	No offline mode available
grep (L4)	✓ Yes	Built-in — always available
Bandit	✓ Yes	None
ShellCheck	✓ Yes	None
cppcheck	✓ Yes	None
hadolint	✓ Yes	None
checkov	✓ Yes	None — local checks only
ScanCode	✓ Yes	None — all licence texts bundled

Offline preparation script

Run this once on a machine with internet access, then copy /opt/ai-transit/offline-cache/ to the air-gapped host:

```
#!/usr/bin/env bash
# prepare_offline_cache.sh
CACHE="/opt/ai-transit/offline-cache"
mkdir -p "${CACHE}/semgrep-rules" "${CACHE}/trivy-db"

echo "[1/3] Downloading Semgrep rules..."
semgrep --config p/owasp-top-ten --dump-config > "${CACHE}/semgrep-rules/owasp.yaml"
semgrep --config p/cwe-top-25 --dump-config > "${CACHE}/semgrep-rules/cwe.yaml"
semgrep --config p/security-audit --dump-config > "${CACHE}/semgrep-rules/audit.yaml"
semgrep --config p/secrets --dump-config > "${CACHE}/semgrep-rules/secrets.yaml"

echo "[2/3] Downloading trivy DB..."
TRIVY_DB_REPOSITORY=ghcr.io/aquasecurity/trivy-db \
trivy image --download-db-only --cache-dir "${CACHE}/trivy-db"

echo "[3/3] Copying ClamAV DB..."
cp /var/lib/clamav/*.cvd "${CACHE}/" 2>/dev/null || \
cp /var/lib/clamav/*.cld "${CACHE}/" 2>/dev/null || true

echo "Done. Copy ${CACHE} to the offline host."
```

11. Sample Scans — Testing the Pipeline

11.1 Quick smoke test (local directory)

Create a small test repository to validate the pipeline end-to-end:

```
#!/usr/bin/env bash
# create_test_repo.sh – creates a clean sample repo that should PASS

mkdir -p /tmp/sample-repo/{src,tests,infra}

# Python source
cat > /tmp/sample-repo/src/app.py << 'EOF'
import os
import sqlite3

def get_user(db_path: str, username: str) -> list:
    """Retrieve user using parameterised query (safe)."""
    conn = sqlite3.connect(db_path)
    return conn.execute(
        "SELECT * FROM users WHERE name = ?", (username,)
    ).fetchall()

def main():
    db = os.environ.get("DB_PATH", "/tmp/app.db")
    print(get_user(db, "alice"))
EOF

# Shell script
cat > /tmp/sample-repo/src/setup.sh << 'EOF'
#!/usr/bin/env bash
set -euo pipefail
APP_DIR="${APP_DIR:-/opt/app}"
mkdir -p "$APP_DIR"
echo "Setup complete"
EOF

# Requirements file (up-to-date)
cat > /tmp/sample-repo/requirements.txt << 'EOF'
requests==2.32.3
Pillow==10.3.0
EOF

# README
echo "# Sample project" > /tmp/sample-repo/README.md

echo "Test repo created at /tmp/sample-repo"
```

```
bash create_test_repo.sh
./ai_transit.sh /tmp/sample-repo
# Expected: PASS – ZIP produced in ./Good/
```

11.2 FAIL test — embedded secret

```
mkdir -p /tmp/fail-secret
```

```

cat > /tmp/fail-secret/config.py << 'EOF'
# Bad practice: hardcoded credential
AWS_ACCESS_KEY_ID = "AKIAIOSFODNN7EXAMPLE"
AWS_SECRET_ACCESS_KEY = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
EOF

./ai_transit.sh /tmp/fail-secret
# Expected: FAIL – betterleaks or detect-secrets detects the AWS keys
# Files moved to /opt/ai-transit/quarantine/

rm -rf /tmp/fail-secret

```

11.3 FAIL test — SQL injection (OWASP A03)

```

mkdir -p /tmp/fail-sqli

cat > /tmp/fail-sqli/db.py << 'EOF'
import sqlite3

def login(username, password):
    conn = sqlite3.connect("app.db")
    query = "SELECT * FROM users WHERE name='" + username + \
    "'" AND pass='" + password + "'"
    return conn.execute(query).fetchall()
EOF

./ai_transit.sh /tmp/fail-sqli
# Expected: FAIL – Semgrep detects SQL injection (OWASP A03 / CWE-89)

rm -rf /tmp/fail-sqli

```

11.4 FAIL test — vulnerable dependency (CVE)

```

mkdir -p /tmp/fail-cve

cat > /tmp/fail-cve/requirements.txt << 'EOF'
Pillow==9.0.0
requests==2.18.0
EOF

./ai_transit.sh /tmp/fail-cve
# Expected: FAIL – trivy/pip-audit detect CVEs in Pillow 9.0.0

rm -rf /tmp/fail-cve

```

11.5 WARN test — risky licence (GPL)

```

mkdir -p /tmp/warn-licence

cat > /tmp/warn-licence/main.py << 'EOF'
# This file is licensed under the GNU General Public License v3.0
# Copyright (C) 2024 Example Corp.
# You may redistribute and/or modify under the terms of the GPL.

def hello():
print("hello")
EOF

./ai_transit.sh /tmp/warn-licence
# Expected: PASS (WARN logged) – ScanCode detects GPL-3.0 licence
# ZIP produced in ./Good/ but WARN logged for legal review

rm -rf /tmp/warn-licence

```

11.6 FAIL test — Dockerfile misconfigurations

```

mkdir -p /tmp/fail-docker

cat > /tmp/fail-docker/Dockerfile << 'EOF'
FROM ubuntu:latest
ENV DB_PASSWORD=mysecretpassword
RUN apt-get update && apt-get install -y curl
ADD https://example.com/app.tar.gz /app/
EOF

./ai_transit.sh /tmp/fail-docker
# Expected: FAIL – hadolint detects :latest tag, ENV secret, ADD from URL

rm -rf /tmp/fail-docker

```

11.7 Scan a real public GitHub repository

```

# Scan a small, well-known public repo (adjust URL as needed)
./ai_transit.sh https://github.com/psf/requests main
# Expected: PASS or WARN – clean repo, dependencies may have minor CVEs

```

11.8 Run the self-check to verify the pipeline itself

```

python3 selfcheck.py \
--bundle-dir . \
--output selfcheck_report.pdf

# Expected output:
# §11.1 Meta-scan → PASS or WARN
# §11.2 Binary checks → SKIP (unless --checksums provided)

```

```
# §11.3 GPG/cosign → PASS or WARN
# §11.4 Python CVE → PASS
# §11.5 Host OS CVE → PASS or WARN
# §11.6 Bundle integrity → PASS
# §11.7 AIDE → SKIP (unless AIDE installed)
```

12. Self-Scan: Verifying the Installation is Safe

Before deploying in a production environment, run the full self-check:

```
python3 selfcheck.py --bundle-dir . --output selfcheck_report.pdf
```

This executes all §11 checks (meta-scan, binary checksums, GPG/cosign, Python CVE scan, host OS CVE, bundle integrity, AIDE) and produces a colour-coded PDF report. See `selfcheck.py --help` for all options.

13. Security Hardening Recommendations

Dedicated service account

```
sudo useradd -r -m -d /opt/ai-transit -s /bin/bash aitransit
sudo chown -R aitransit:aitransit /opt/ai-transit
sudo -u aitransit ./ai_transit.sh https://github.com/org/repo
```

Network isolation (allow only github.com outbound)

```
sudo ufw default deny outgoing
sudo ufw allow out 443 comment "HTTPS for GitHub"
sudo ufw allow out 53 comment "DNS"
sudo ufw enable
```

Quarantine filesystem (noexec)

```
# /etc/fstab
tmpfs /opt/ai-transit/quarantine tmpfs rw,noexec,nosuid,nodev,size=2G 0 0
```

Weekly tool updates (cron)

```
# /etc/cron.weekly/ai-transit-update
#!/bin/bash
freshclam
trivy image --download-db-only
source /opt/ai-transit/venv/bin/activate
pip install --upgrade betterleaks detect-secrets semgrep bandit \
pip-audit safety checkov scancode-toolkit
```

Log rotation

```
# /etc/logrotate.d/ai-transit
/opt/ai-transit/logs/*.log {
daily
rotate 30
compress
missingok
notifempty
}
```

14. Troubleshooting

Symptom	Likely cause	Fix
betterleaks: command not found	Binary not in PATH	<code>sudo cp ~/go/bin/betterleaks /usr/local/bin/</code>
freshclam: connect refused	clamav-freshclam not running	<code>sudo systemctl start clamav-freshclam</code>
semgrep: network error	No internet — first run	Pre-download rules (see §10)
trivy: DB not found	First run without internet	<code>trivy image --download-db-only</code>
scancode: takes too long	Large repo	Use <code>--timeout 30</code> to limit per-file time
pip-audit: no vulnerabilities on old packages	DB unreachable	Check internet; use trivy offline instead
npm audit: ENOLOCK	No package-lock.json	Run <code>npm install --package-lock-only first</code>
openpyxl manquant	Not installed in venv	<code>pip install openpyxl</code>
declare -A: invalid option	Bash < 4.0	Install bash 5: <code>brew install bash</code> / use WSL2
zip: command not found	zip not installed	<code>sudo apt-get install zip</code>
WARN on all files	Missing optional tools	Install missing tools; WARNs do not block
Git clone fails	Private repo	Only public GitHub repos are supported

Quick Reference — One-Line Install (Ubuntu 22.04 / 24.04)

```
# Step 1: system packages
sudo apt-get update && sudo apt-get install -y \
bash git curl jq zip unzip file coreutils wget gpg \
python3 python3-pip python3-venv build-essential golang-go \
nodejs npm shellcheck cppcheck clamav clamav-daemon yara
```

```

# Step 2: Python venv + packages
python3 -m venv /opt/ai-transit/venv
source /opt/ai-transit/venv/bin/activate
pip install --upgrade pip
pip install openpyxl reportlab detect-secrets bandit pip-audit safety \
semgrep checkov scancode-toolkit

# Step 3: betterleaks
go install github.com/betterleaks/betterleaks@latest
sudo cp ~/go/bin/betterleaks /usr/local/bin/betterleaks

# Step 4: trivy
curl -sFL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh \
| sudo sh -s -- -b /usr/local/bin

# Step 5: hadolint
VERSION=$(curl -s https://api.github.com/repos/hadolint/hadolint/releases/latest \
| jq -r '.tag_name')
sudo curl -sSL \
"https://github.com/hadolint/hadolint/releases/download/${VERSION}/hadolint-Linux-x86_64" \
-o /usr/local/bin/hadolint && sudo chmod +x /usr/local/bin/hadolint

# Step 6: ClamAV DB update
sudo freshclam

# Step 7: trivy DB
trivy image --download-db-only

echo "Installation complete – run ./verify_install.sh to confirm"

```